Generative AI Systems
Multimodal Generative AI
VQ-variational

1. Introduction and Overview
2. Vision Small Transformer
3. Image Tokenizers
4. DaLL·E Transformer
5. Image Tokenizer
6. Image Captioning
7. Multimodal Augmented Generation 1
8. Parallelizing Text Generation
9. Text-to-Image Generation
10. Prompt and Response Multimodal Generation
11. Text-to-Video Generation

09
Text-to-Image Generation

Introduction

Figure 1: An example of a prompt and a generated image by OpenAI's DALL-E 2[1].

Clarifying Requirements

Here is a type of interaction between a candidate and an interviewer:

Candidate: What evaluation do we target for generated images?
Interviewer: We can use the inception score, specifically FID(Fréchet distance).

Candidate: Should the system support multiple languages for input or just English?
Interviewer: Let's keep it simple with English. Multilingual support is a desirable extrapolation for other languages later.

Candidate: Any bias in the dataset I'm training on that I need to account for?
Interviewer: We have about 600 million images from crawl datasets, bound with captions.

Candidate: How detailed and concepts are the captions? Are they written by other crowd-workers? Or single?
Interviewer: The system should handle detailed text prompts, with a maximum length of 128 words.

Candidate: What about latency of the system, should it be real-time?
Interviewer: The goal is near real-time generation. Let's aim for 15 seconds on average.

Candidate: Do we have any constraints on the hardware? How many machines, specifically, shows the hardware?
Interviewer: Let's assume we have access to 256 A100 GPUs of gradually, based on last photo, 40 gigabyte vRAMs, bandwidth, latencies, permalink, and absolute or consequential set.

Candidate: It's important to know the system's intent biased by age, race, or gender. Can I start by focusing on those bias amplifications?
Interviewer: Great point. It's crucial to have a fair system. Let's begin by addressing those three attributes.

Candidate: Understand. Mitigations are crucial. We need filters and checks to avoid generating offensive, inappropriate, or harmful images. Does that sound correct?
Interviewer: Yes, that's the goal.

Frame the Problem as an ML Task

Specifying the system's input and output

The task is to take a text prompt and generate an image that describes the described image. This prompt usually includes a main subject, a style, and modifiers (such as adjectives, colors, and lighting).

The output is a visually detailed image that adheres to the text prompt. For example, as shown in Figure 2, a prompt like "a lion on a cliff" produces an image depicting this scene.

Figure 2: Input and output of a text-to-image system. Image credit: [2].

Choosing a suitable ML approach

Text-to-image generation is a multimodal task that involves understanding text and generating a corresponding image. There are two primary approaches for training such a system:

- Autoregressive models
- Diffusion models

Let's briefly review each and choose the one that best suits our needs.

Autoregressive models

These models treat text to image generation as a sequence generation task. A decoder-only Transformer takes a sequence of text tokens as input and predicts a sequence of visual tokens representing an image. An image tokenizer then decodes these visual tokens into the actual image.

Figure 3: Autoregressive text-to-image generation.

Several text to image models have been developed using this approach, such as DaleRM (Wav-LLM) and DaLL·E (and Google's Parti).

Diffusion models

Diffusion models offer an alternative approach. They learn to generate images by iteratively refining random noise guided by the text prompt. The process typically begins with a random noise tensor and progressively refines it over many steps, guided by the text prompt. These refinements gradually transform the noise into a coherent image that aligns with the text prompt. Finally, a decoder-only Transformer then upsamples the refined image to the final output resolution.

Figure 4: Diffusion text-to-image generation.

Examples of diffusion based text-to-image models include Google's Imagen, DALL·E(Ada), and Stability AI's Stable Diffusion[10].

Diffusion versus autoregressive models

Autoregressive models frame text-to-image generation as a sequence generation task, while diffusion models approach it as an iterative refinement process. The choice between these models involves trade-offs:

Figure 5: Diffusion vs autoregressive model architectures.

- **Implementation complexity:** Autoregressive models can provide simpler to implement during both training and inference. Training involves standard next token prediction. Inference involves sampling from the output probability distribution. Diffusion models, in contrast, diffusion models are less relatively efficient, requiring sampling of different noise levels for each image. Implementing the iterative denoising process and managing the different stages of the diffusion process adds complexity.
- **Inference speed:** Generating images from text using autoregressive models can be slow, as it requires sequentially predicting tokens.
- **Image quality:** Diffusion models have shown better performance in generating highly detailed and realistic images. They excel at capturing complex details and textures, which is crucial for photo-realistic scenes. Ensuring consistent quality across the entire image can be more challenging for autoregressive models.
- **Controllability:** Diffusion models offer more flexibility in making adjustments or refinements to specific parts of the image without regenerating the entire image. This allows for fine-grained control over the generation process. Controlling the attributes of the generated image in autoregressive models might involve more complex techniques like conditional sampling or fine-tuning.

In this chapter, we choose diffusion models to prioritize computational image quality, in the model development section, we explore the architecture, training process, and sampling mechanisms of diffusion models.

Data Preparation

Our dataset consists of approximately 600 million image-caption pairs. However, large scale datasets often require substantial preprocessing to ensure data quality. We can leverage a pre-trained CLIP model to filter the dataset, we explore curation techniques to prepare images and captions for efficient training.

Image preparation

We focus on two main steps to prepare the images: filtering inappropriate images and standardizing the remaining ones. Let's delve into each step in detail.

Filtering inappropriate images

A large scale dataset, likely crawled from the web, can contain many inappropriate or low-quality images. These images can negatively impact the model's learning process. We employ several filtering techniques:

- **Remove small images:** We discard images captions pairs where the image size is smaller than a certain threshold, for example, 256 pixels. Small images are often of poor quality and may not provide valuable information for training.
- **Inadequate images:** We use datasets-inference, such as CLIP, to remove identical or perceptually similar images. This ensures diversity in the dataset and prevents the model from overfitting to specific images or objects.
- **Remove inappropriate images:** We employ binary classifiers trained on NSFW (Not Safe For Work) datasets to identify and filter out harmful content such as violence or nudity. This ensures the model doesn't learn to generate inappropriate outputs.
- **Aesthetic quality filtering:** We use a pre-trained aesthetic quality classifier to evaluate the remaining images. This allows us to filter out images with low aesthetic scores, ensuring the model learns to generate visually pleasing outputs.

Standardizing images

- **Aspect-image dimensions:** We utilize random square crops of specific dimensions. Therefore, it's crucial to have training data of similar sizes. For example, if the expected model input is 128×128, we first resize the images so that the smaller dimension matches the corresponding dimension of the crop, then perform a random crop to extract a square image.
- **Normalize images:** We normalize pixel values to the standard range such as [-1, 1] or [0, 1] for more stable training.

Figure 6: Image preparation steps.

Caption preparation

In addition to images, captions are often crawled from the internet and require cleaning to ensure captions are consistent and of high quality.

- **Handle missing or non English captions:** For images without captions or with captions in another language, we can generate synthetic captions using a captioning model. This ensures all training data contributes to the model's understanding of image-caption relationships.
- **Enhance captions:** We use a pre-trained model such as CLIP [8] to score the relevance of each image-caption pair. For pairs with low scores, we can generate new captions using a captioning model to improve the quality of the training data.
- **Remove poorly matchedimages:** After enhancing captions, we remove image-caption pairs with CLIP similarity scores below a threshold. This step ensures that the model is exposed only to pairs where captions accurately describe the images.

Model Development

Architecture

A diffusion model, as previously explained, progressively denoises a noisy image through multiple steps until it becomes clear. In practice, we train a model to predict the noise added to a clean image at each step and provides the noise to be removed.[1] The underlying architecture is a U-Net.

- U-Net
- Diffusion Transformer (DiT)

Figure 7: Overview of the diffusion process.

U-Net

U-Net [10] is a convolutional neural network (CNN) architecture originally developed for biomedical image segmentation. It consists of a series of downsampling blocks followed by upsampling blocks, as shown in Figure 8.

Figure 8: U-Net architecture with residual blocks and attention blocks.

Downsampling blocks:

The downsampling blocks progressively reduce spatial dimension (height and width) while increasing depth (number of channels), leading to a compressed representation of the input. Each downsampling block typically consists of the following:

- **Residual connections:** Residual connections ease the flow of information.
- **Attention layers:** Attention layers enable the model to focus on relevant parts of the input.
- **Non-linear activation:** Introduces non-linearity to learn complex patterns.
- **Pooling or strided convolution:** Reduces spatial dimensions.

The bottleneck block processes the compressed representation from the downsampling blocks. It's typically a series of residual blocks with attention layers, ensuring the model captures long-range dependencies and complex interactions between different parts of the image.

Upsampling blocks:

The upsampling blocks gradually increase spatial dimensions and decrease feature depth, aiming to reconstruct a high-resolution image from the compressed representation. Each upsampling block typically includes:

- **Transposed convolution:** Uses operations like PyTorch's ConvTranspose2d to process and increase the feature maps dimensions.
- **Batch normalization:** Normalizes feature maps to stabilize training.
- **Non-linear activation:** Introduces non-linearity to learn complex patterns.
- **Cross-attention:** Maintains the influence of additional conditioning information.

Figure 9: U-Net architecture with cross-attention blocks.

The U-Net architecture has various variants and modifications. Different implementations may use different layers and configurations. However, understanding the key components and structure is generally sufficient for most ML system design interviews. For more in-depth information, refer to [12].

DiT

DiT [16] is another popular architecture for diffusion models. Unlike U-Net, which uses a series of downsampling and upsampling layers, DiT primarily relies on a Transformer architecture to process noisy image and output the noise.

Figure 10: DiT architecture.

DiT is primarily inspired by the Vision Transformer (ViT) architecture discussed in Chapter 5. DiT components are:

- **Patchify:** Converts the input image into a sequence of patch embeddings.
- **Transformer blocks:** Process the patch embeddings, incorporating conditioning information such as timestep and class embeddings.
- **Transformer:** Processes the sequence of transformer output tokens into an image with the same dimension as the original input image.

In summary, both U-Net and DiT architectures effectively process noisy images and predict the noise. The choice between these models can depend on factors like computational resources and the specific requirements of the application. For educational purposes, we use the U-Net architecture in this chapter. Chapter 11 explores the DiT architecture in more detail.

Training

A diffusion model is trained by employing the diffusion process. The diffusion process has two phases:

- Forward process
- Reverse process

Forward process

In the forward process, noise is added to the **training process**, noise is gradually added to an image over multiple steps (denoted as t timesteps) until the image becomes completely noisy. The value of t representing the number of steps, is typically chosen randomly from a large number (for example, 1000). This ensuring the model learns to handle various levels of noise in the training process.

Figure 11: Forward diffusion process.

Backward process

In the backward process, also known as the **denoising process**, an ML model learns to reverse the forward process. At each step, the model predicts the noise added to the image at that step. The goal is to accurately reduce the noise in the noisy image. As shown in Figure 12, this process ultimately results in a clean image.

Figure 12: Backward diffusion process.

With an understanding of both the forward and backward processes, we can explore how they are applied during the diffusion training process.

Diffusion training process

During training, we introduce noise to the original image by simulating the forward process. Even ask the model to predict this noise. The process involves four steps:

1. Noise addition
2. Time step conditioning signals
3. Noise prediction
4. ML objective and loss calculation

This process includes some terms that you are interested, but the details do not affect the design of the ML system.

1. Noise addition

The first step is to simulate the forward process by adding noise to the original image over multiple timesteps. At each timestep, we corrupt the image slightly by adding Gaussian noise. This gradual addition of noise transforms the image into pure noise over time.

The amount of noise added at each timestep is controlled by a **noise schedule**. The noise schedule is defined by a set of variance parameters, β_t, where t represents the timestep. These variance parameters dictate the amount of noise added at timestep t.

The noise schedule typically increases the value of β monotonically:

$$\beta_1 < \beta_2 < \dots < \beta_T$$

This ensures that more noise is added at the early steps, preserving more information of the original image, and larger amounts of noise are added to the later steps, ensuring the information is mostly noise.

With the noise schedule defined, we can express the noisy version of timestep t by using the noise schedule formula:

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \epsilon_{t-1}$$

where:

- x_t is the noisy image at timestep t.
- x_{t-1} is the noisy image at timestep $t-1$.
- β_t is the variance parameter at timestep t from the standard normal distribution $\mathcal{N}(0, I)$.
- ϵ_{t-1} is the random Gaussian noise sampled from the standard normal distribution $\mathcal{N}(0, I)$.

Iteratively applying noise over many timesteps can lead to numerical instability. To address this issue, we can reformulate the equation to express x_t directly derived from the original image x_0:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$$

$$\bar{\alpha}_t = \prod_{s=1}^{t} (1-\beta_s)$$

$$\epsilon \sim \mathcal{N}(0, I)$$

where:

- x_t is the noisy image at timestep t.
- x_0 is the original image.
- $\bar{\alpha}_t$ is a cumulative product of $(1-\beta_s)$ for all timesteps from 1 to t.
- ϵ is the Gaussian noise sampled from the standard normal distribution $\mathcal{N}(0, I)$.

In summary, during noise addition, we add noise to images at various timesteps, directly from x_0, without the need to iteratively add noise at each timestep, using the following formula:

$$x_t = \text{add_noise}(x_0, \epsilon, t)$$

This ensures stable and efficient noise addition.

To parallelize the process, we create equally spaced but additional groups of timestamps: the image samples and the sampled timesteps, t which indicates the noise level. We use separate encoders (see Figure 16) to prepare each of these conditioning signals before combining them.

Figure 16: Image encoder and noise level encoder.

Image encoder

The image encoder processes the noisy image to prepare it for the main diffusion process. Its role is to extract relevant features while preserving the spatial information. The encoder typically consists of convolutional layers or a small U-Net architecture. The output of the image encoder is a set of feature maps that capture the essential information from the noisy image.

Noise level encoder

The noise level encoder processes the timestamp t, which indicates the amount of noise added to the image. This encoder transforms the timestamp into a vector representation that the model can understand. This vector representation typically includes sinusoidal embeddings or learned embeddings, allowing the model to learn the optimal way to represent different noise levels.

Combine and process

The outputs of the image encoder and the noise level encoder are then combined, typically by concatenation or addition. This combined representation serves as input to the main diffusion model, which then processes this information to predict the noise added to the original image at timestamp t.

2. Time step conditioning signals

To enable the model to handle various noise levels, we incorporate additional signals to provide information about the current timestep. t which indicates the noise level. We use separate encoders (see Figure 16) to prepare each of these conditioning signals before combining them.

3. Noise prediction

The primary goal of training a diffusion model is to predict the noise added to the forward diffusion process that, is, to reconstruct the original data, x_0, from the noisy version, x_t. A key been shown that directly predicting x_0 is not as effective. Instead, training the model to predict the noise, ϵ, added to the image at each timestep, t, yields better results. This approach stabilizes the training process and allows the model to learn the underlying x_0 given the noisy image, x_t and the timestep, t.

4. ML objective and loss calculation

The ML objective is to minimize the difference between the true noise, ϵ, and the model's prediction. The loss function used is the mean squared error (MSE) between the true noise and the predicted noise:

$$L = \mathbb{E}_{x_0, \epsilon, t} \left[\| \epsilon - \epsilon_\theta(x_t, t) \|^2 \right]$$

where:

- L is the loss.
- $\mathbb{E}_{x_0, \epsilon, t}$ is the expectation over x_0, ϵ, and t.
- ϵ is the Gaussian noise sampled from the standard normal distribution $\mathcal{N}(0, I)$.
- $\epsilon_\theta(x_t, t)$ is the predicted noise by the model, parameterized by θ, given the noisy image x_t and the timestep t.

In summary, during noise addition, we add noise to images at various timesteps, directly from x_0, without the need to iteratively add noise at each timestep, using the following formula:

$$x_t = \text{add_noise}(x_0, \epsilon, t)$$

This ensures stable and efficient noise addition.

To parallelize the process, we create equally spaced but additional groups of timestamps: the image samples and the sampled timesteps, t which indicates the noise level. We use separate encoders (see Figure 16) to prepare each of these conditioning signals before combining them.

Figure 16: A single iteration in diffusion training.

To enhance the training experience, we can leverage the mathematical details, such as the derivation of the Variable mean and variance, and implement techniques such as Variable lower bound (VLB) optimization.

Sampling

Generally, training is generating a new image from a trained diffusion model. In this section, we'll explore how sampling works in diffusion models and how noise-prediction and time-conditioned neural networks work together to generate high-quality images.

The sampling process begins with a random noise tensor, sampled from the standard Gaussian distribution. The model then iteratively refines this noisy input, guided by the learned noise prediction and the current timestep. At each step, the model predicts the noise present in the image (Figure 16) and adjusts the image slightly in the right direction. This gradual refinement continues, each step producing a cleaner image until a clear and detailed image is achieved.

Figure 15: Sampling process.

The basic sampling procedure described above has two drawbacks. First, it often fails to generate images that are perceptually realistic (as perceptual). Second, it is slow, because generating each sample requires many iterated steps. The following two techniques are proposed to address these drawbacks and improve sampling efficiency.

- **Classifier-free guidance (CFG):** CFG [3] improves the alignment between image and text prompts in diffusion models. During training, the model learns to generate images with and without the text prompt. During sampling, CFG adjusts the noise prediction to amplify the influence of the text prompt. This guidance steers the model towards generating images more closely aligned with the text description, enhancing the controllability of the generation process. The guidance scale parameter controls the trade-off between diversity and adherence to the prompt. To learn more about CFG, refer to [3].
- **Latent diffusion models:** Instead of operating directly on pixel values, latent diffusion models [10] perform the diffusion process in a lower-dimensional latent space. This significantly speeds up the sampling process and reduces computational resources. For more details, see Section 11.1.1.

Scaling text-to-image diffusion models presents several challenges during both training and sampling. The most common ones include:

- Resource intensive model training
- Slow inference generation

Resource intensive model training

Training diffusion models is computationally intensive, requiring significant processing power. It also requires substantial GPU memory to accommodate the model and its gradients. Techniques like mixed precision training can mitigate memory issues, but still pose a challenge. Optimizing the training process is crucial for efficient resource utilization and potential cost reduction. The following strategies are commonly employed to overcome these challenges:

- **Mixed-precision training:** This technique uses both 16-bit and 32-bit floating point numbers to balance memory usage and increase computational efficiency. To learn more, refer to [20].
- **Distributed training:** Distribute the training process across multiple devices. When distributed training frameworks such as PyTorch DDP and Horovod offer support, consider implementing them.
- **Latent diffusion models:** These models operate in a lower-dimensional latent space, significantly reducing memory and computational requirements. Chapter 11 provides more details on this approach.

Slow image generation

The iterative nature of diffusion models is slow for real-time scenarios. First, due to the sequential nature of the diffusion process, multiple sampling steps are needed to refine the image. Second, the diffusion process operates on high-dimensional data, making each step computationally intensive.

Several strategies can address this challenge:

- **Parallel sampling:** Implementing parallel processing during sampling reduces the time needed to generate images [21].
- **Distillation:** Distillation techniques can significantly reduce the number of sampling steps required. This involves training a smaller, faster model to mimic the behavior of the original model while maintaining its performance. Chapter 11 explores this in more detail.

- **Model quantization:** This technique reduces the precision of the model's weights, which decreases memory usage and speeds up generation.

Evaluation

Offline evaluation metrics

A consistent and reliable benchmark is key to evaluating text-to-image models. Drawbench [57] serves this purpose by providing a curated set of prompts that test various aspects of image generation such as object composition, interaction, and context understanding. These prompts, ranging from simple to complex, help in gauging how accurately the model generates images from text. Due to its comprehensiveness, we use Drawbench to evaluate our text-to-image model. Let's examine both automated metrics and human evaluation to assess three key areas of the model's capabilities in generation:

- Image quality
- Image diversity
- Image-text alignment

As discussed in previous chapters, Inception score (IS) [27] and Fréchet Inception distance (FID) [28] are two common metrics for evaluating quality and diversity in image generation systems. In this section, we focus primarily on image-text alignment.

Image-text alignment

Image-text alignment refers to how accurately the generated images match the text prompts. Measuring this alignment is important because it ensures the generated images are faithful to the prompt's intent. A common metric for assessing this is CLIPScore [29], which evaluates the degree of alignment. Before diving into CLIPScore, let's briefly review CLIP.

CLIP

CLIP [8] is a model developed by OpenAI that has been trained to match images with their corresponding descriptions. It consists of two encoders: one for text and one for images. The text encoder converts input text into a text embedding, the image encoder converts the image into one embedding.

Figure 16: CLIP encoders

During training, CLIP learns to align embeddings by bringing related text and image embeddings closer together and pushing unrelated ones apart. This helps CLIP develop a shared embedding space where both an image and its associated text will map into the same space.

After training, similar text descriptions map close to each other in the embedding space, and images may near their relevant descriptions.[1]

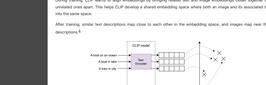

Figure 17: Text and image feature vectors mapped to the CLIP embedding space

With an understanding of the CLIP model, we can now easily explore CLIPScore as a metric for evaluating image-text alignment.

CLIPScore

The CLIPScore measures the cosine similarity between the CLIP embeddings of a text description and an image. It shows how closely the image matches the text in the high-dimensional embedding space. A higher score indicates better alignment between an image and a description.

Human evaluation

Human evaluation complements the automated metrics by assessing both image quality and text alignment using the following approach:

- **Image quality:** Human raters compare a generated image with a reference image by judging which of the images is more photorealistic. Image quality is determined by the percentage of times the generated image is chosen over the reference image.
- **Text alignment:** Human raters are shown an image and its caption and then asked, "Does the caption accurately describe the above image?" The responses "yes," "somewhat," and "no" are scored as 100, 50, and 0, respectively. These scores are averaged separately for generated images and reference images to measure text alignment.

Online evaluation metrics

Online metrics measure how the model works in production. Common metrics to evaluate our text-to-image model include:

- **Click-through rate (CTR):** The percentage of users who click on the generated images. A high CTR indicates that users find the generated images useful.
- **Time spent on page:** The average time users spend on the service. Longer viewing times indicate higher user engagement.
- **User feedback:** Direct feedback from users is collected through feedback. Positive feedback indicates satisfaction with the image quality and text alignment.
- **Conversion rate:** The percentage of users who take a desired action (e.g., purchase, sign up) after interacting with generated images. A high conversion rate indicates satisfaction with the model's performance.
- **Latency:** The time it takes to generate an image from a text prompt. Lower latency indicates faster performance, which is important for user satisfaction.
- **Queries per second (QPS):** The number of image generations the model can handle per second. High throughput ensures the service can scale effectively.
- **Resource utilization:** The computational resources (e.g., CPU, GPU, memory) used to run the model and serve users. Efficient resource utilization is key to reducing costs.
- **Average cost per user per month:** Generating images with models that have billions of parameters is costly. If users are unhappy with the images, they might repeatedly generate new ones using the same prompt but different seeds, hoping for better results. This behavior increases our expenses. By monitoring this metric, we can ensure that costs remain justifiable.

Overall ML System Design

While the diffusion model is at the core of a text-to-image generation system, several other pipelines are crucial for ensuring efficiency, safety, and quality. In this section, we delve into the holistic design of a text-to-image generation system by examining the following pipelines:

- Data pipeline
- Training pipeline
- Model optimization pipeline
- Inference pipeline

Data pipeline

The data pipeline prepares data for training by removing inappropriate images, standardizing the rest, and storing them. It ensures captions are present and relevant and uses a pretrained model to generate synthetic captions if needed. It also pre-computes and caches caption embeddings. This caching reduces computation during training.

In addition to preparing text-image pairs from the training data, the pipeline also collects and processes newly generated data such as user prompts, generated images, and user feedback. This new data is added to the training set for future use.

Figure 18: Data pipeline

Training pipeline

The training pipeline trains a model using the latest training data collected by the data pipeline.

Figure 19: Training pipeline

The training pipeline ensures that the model adapts to recent user prompts and is trained on higher-quality generated images.

Evaluation pipeline

The evaluation pipeline assesses newly trained models using predefined automated metrics to determine whether or not they meet performance and quality standards for deployment.

Model optimization pipeline

The model optimization pipeline is responsible for enhancing the efficiency of the model. There are several methods to optimize models.

- **Model compression:** Use techniques such as quantization and pruning to reduce model size and generation time.
- **Model distillation:** Distill the model into a smaller one to reduce the model size and generation time.
- **Optimized algorithms:** Replace sampling with more efficient algorithms for faster generation.

Once the model optimization is completed, the optimized model can replace the existing model in production.

Inference pipeline

The inference pipeline handles user requests and generates images based on text prompts. It comprises several components, each playing an important role in ensuring the system's quality and safety. In this section, we will examine the key components and their functions:

- Prompt auto-complete
- Prompt safety service
- Prompt enhancement
- Image generation
- Harm detection
- Super-resolution service

Figure 20: Inference pipeline

Prompt auto-complete service

The prompt auto-complete service uses a specialized model to suggest possible next words or phrases in real-time as the user types their prompt. This enhances user experience by providing potential completions.

Figure 21: Suggested phrases by the auto-complete service

Prompt safety service

This service uses a text classification model to process user prompts and reject those that violate our usage policies, for example, requests for violence, hateful imagery, or nudity.

This service ensures that the system adheres to safety standards and prevents the generation of inappropriate images.

Prompt enhancement

The prompt enhancement component refines user prompts to improve their clarity, coherence, and details.

Figure 22: An example of prompt enhancement

This component is widely used in advanced image and video generation systems [35], as it effectively helps the model produce better outputs. It improves the quality of generated images by offering the model a more coherent and detailed prompt.

Image generation

The image generation component is the core of the inference pipeline. It interacts with the T5 text encoder to encode the enhanced text prompt into a sequence of tokens. These tokens are passed to the diffusion model to generate one or multiple images for each prompt.

Harm detection

This component ensures that generated images are safe for users. If an image still contains violence or nudity despite previous safeguards, the component flags it and blocks it from being shown.

Super-resolution service

The super-resolution service increases the resolution of the generated images. This step ensures that the final output is visually appealing and meets resolution requirements.

In practice, text-to-image systems often use at least one super-resolution model because diffusion models typically cannot generate high-resolution images directly. Instead, the diffusion model is trained at a lower resolution, and specialized super-resolution models enhance the resolution. For example, the base model might generate a 64x64 image, which a first super-resolution model increases to 256x256, followed by a second model that boosts it to 1024x1024. Google's [37] follows this approach to achieve the desired resolution.

Figure 23: A cascader super-resolution models

In summary, various pipelines work together to ensure that a text-to-image system is reliable, high-quality, and safe. The data pipeline provides the foundation for continuous improvements, while the training and model optimization pipelines enhance the model's performance. The inference pipeline ensures safe, efficient, and high-quality image generation. These pipelines create a system ready for real-world challenges.

Other Talking Points

If time permits at the end of the interview, consider discussing these additional topics:

- Using consistency models for faster image generation [36].
- Employing RLHF for quality improvement [20].
- Extending the text-to-image model to support inpainting and outpainting applications [37].
- Personalizing the text-to-image model to a particular concept (Dreambooth).
- Details of different scheduling techniques [34].
- Details of the U-Net architecture and other architectural foundations [20][38].
- Supporting multiple aspect ratios and resolutions using techniques such as Patch n' Pack [39].
- Details of developing a re-captioning model [36][37][10].
- Improving the diversity-fidelity trade-off with guidance [38].
- More advanced controls over the generated images using techniques like ControlNet [38].
- Controlling the style of the generated images [38].

Summary

Reference Material

[1] Davis's DALL-E 3. https://cdn.openai.com/dall-e-3/ [2] Imagen 3. https://arxiv.org/abs/2408.07009
[3] Google's Parti. https://www.assistive.com/blog/parti/#parti-text
[4] Introducing DreamFusion. https://dreamfusion3d.github.io/ [5] Zero-Shot Text-to-Image Generation. https://arxiv.org/abs/2102.12092
[6] Made Text-To-Image Generation via Masked Generative Transformers. https://arxiv.org/abs/2301.00704
[7] Generative Modeling by Estimating Gradients of the Data Distribution. https://arxiv.org/abs/1907.05600
[8] Learning Transferable Visual Models From Natural Language Supervision. https://arxiv.org/abs/2103.00020
[9] Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. https://arxiv.org/abs/1910.10683
[10] Hierarchical Text-Conditional Image Generation with CLIP Latents. https://arxiv.org/abs/2204.06125
[11] High-Resolution Image Synthesis with Latent Diffusion Models. https://arxiv.org/abs/2112.10752
[12] On the Importance of Noise in Augmented Neural Ordinary Differential Equations and Guided Diffusion Models. https://openreview.net/pdf?id=Pf2-_ZB5cwO
[13] Elucidating the Design Space of Diffusion-Based Generative Models. https://arxiv.org/abs/2206.00364
[14] Imagen Video: High Definition Video Generation with Diffusion Models. https://arxiv.org/abs/2210.02303
[15] Scaling Rectifier Flow Transformers for High-Resolution Image Synthesis. https://arxiv.org/abs/2403.03206
[16] Scalable Diffusion Models with Transformers. https://arxiv.org/abs/2212.09748
[17] Hierarchical Text-Conditional Image Generation with Diffusion Models. https://arxiv.org/abs/2204.06125
[18] Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding. https://arxiv.org/abs/2205.11487
[19] Denoising Diffusion Probabilistic Models. https://arxiv.org/abs/2006.11239
[20] Classifier-Free Diffusion Guidance. https://arxiv.org/abs/2207.12598
[21] Denoising Diffusion Implicit Models. https://arxiv.org/abs/2010.02502
[22] Introduction to Diffusion Models. https://lilianweng.github.io/posts/2021-07-11-diffusion-models/
[23] Mixed Precision Training. https://arxiv.org/abs/1710.03740
[24] FSDP tutorial. https://pytorch.org/tutorials/intermediate/FSDP_tutorial.html
[25] DeepSpeed. https://github.com/microsoft/DeepSpeed
[26] Prompt Safety Classifier. https://arxiv.org/abs/2305.16311
[27] Consistency models. https://arxiv.org/abs/2303.01469
[28] Inception score. https://en.wikipedia.org/wiki/Inception_score
[29] FID calculation. https://en.wikipedia.org/wiki/Fréchet_inception_distance
[30] CLIPScore: A Reference-free Evaluation Metric for Image Captioning. https://arxiv.org/abs/2104.08718
[31] Stable Diffusion. https://github.com/CompVis/latent-diffusion#stable-diffusion
[32] Imagen Video: High Definition Video Generation with Diffusion Models. https://arxiv.org/abs/2210.02303
[33] Scaling Rectifier Flow Transformers for High-Resolution Image Synthesis. https://arxiv.org/abs/2403.03206
[34] Ablation Studies. https://arxiv.org/abs/2112.10752
[35] StyleGrid: Text-to-image generation in any style. https://research.google/blog/stylegrid-text-to-image-generation-in-any-style/

Footnotes

1. In practice, we provide the diffusion model with additional conditioning inputs, such as the timestep. This will be discussed further in the training section. ↩
2. For simplicity, we omitted the timestep input to the diffusion model. This will be covered in more detail in the training section. ↩
3. For simplicity, we include only the noisy image, x_t, and the timestep, t, as the input. As mentioned earlier, conditioning signals can also be included as inputs. ↩
4. For simplicity, the embedding space is visualized in 2D. In reality, it is a 4-dimensional space, where 4 represents the embedding size. ↩

Share as Complete

Partner With Us
Join an Affiliate
Become a Contributor
Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information
Support
hi@bytebytego.com
Report a Bug
Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare For Coding Interviews
Copyright ©2022-2026 Bytebytegoho Inc. All rights reserved.